

Configuration Google Calendar — SlashBooking

Ce document décrit la mise en place complète de la synchronisation bidirectionnelle entre **SlashBooking** et **Google Calendar** : OAuth (sortant WP → GCal) et watch channel + webhook (entrant GCal → WP).

Pré-requis : plugin `slashbooking` ≥ 1.0.0 installé et activé sur un WordPress accessible en **HTTPS public** (obligatoire pour le webhook Google).

Sommaire

1. Vue d'ensemble
2. Étape 1 — Projet Google Cloud
3. Étape 2 — OAuth consent screen
4. Étape 3 — OAuth 2.0 Client ID
5. Étape 4 — Connexion dans WordPress
6. Étape 5 — Clé de chiffrement (recommandé)
7. Étape 6 — Sync entrante (webhook + pull)
8. Vérification — `wp slashbooking doctor`
9. Diagnostics & monitoring
10. Troubleshooting
11. Désactivation propre
12. Annexe — Scopes & permissions

Vue d'ensemble

SlashBooking synchronise les rendez-vous dans les **deux sens** :

Direction	Mécanisme	Latence
WP → GCal	Action Scheduler <code>sb/push_gcal_event</code>	< 1 min
GCal → WP	Webhook push notifications + cron fallback 15 min	~5 s à 15 min

Côté WordPress, **un seul compte Google** est connecté en V1 (mono-commercial). Tous les RDV créés/confirmés/annulés via le plugin sont reflétés dans ce calendrier, et tout événement créé directement dans Google Calendar **bloque automatiquement** le créneau côté formulaire public.

Étape 1 — Projet Google Cloud

1. Ouvrir [Google Cloud Console](#).
2. Créer un **nouveau projet** (ex : `slashbooking-prod`) ou réutiliser un projet existant.
3. Aller dans **APIs & Services** → **Library**.
4. Rechercher **Google Calendar API** et cliquer **Enable**.

💡 **Astuce** : un seul projet GCP suffit pour tous les environnements (dev, staging, prod) — créer juste plusieurs Client IDs séparés à l'étape 3.

Étape 2 — OAuth consent screen

1. **APIs & Services** → **OAuth consent screen**.
2. **User Type** : **External**.
3. Renseigner : - **App name** : `slashbooking` (ou le nom de l'entreprise). - **User support email** : l'adresse du commercial qui sera connectée. - **Developer contact** : la même adresse.
4. **Scopes** : ajouter manuellement - `https://www.googleapis.com/auth/calendar.events` (lecture/écriture des événements) - `https://www.googleapis.com/auth/calendar.readonly` (liste des calendriers à la connexion)
5. **Test users** : ajouter l'adresse e-mail du commercial.

⚠ En mode **External + Testing**, Google limite à **100 testeurs** et le refresh token expire au bout de **7 jours**. Pour de la prod long-terme, basculer en **Production** (review Google nécessaire si scopes sensibles — Calendar n'en fait pas partie, donc la publication est instantanée).

Étape 3 — OAuth 2.0 Client ID

1. **APIs & Services** → **Credentials** → **+ Create credentials** → **OAuth client ID**.
2. **Application type** : **Web application**.
3. **Name** : `slashbooking - <site>` (ex : `slashbooking - a2c.voilavoila.tv`).
4. **Authorized redirect URIs** → cliquer **+ Add URI** et coller l'URL affichée dans WordPress :

`https://votresite.fr/wp-json/slashbooking/v1/admin/google/oauth/callback`

👉 L'URL exacte est visible dans **WP Admin** → **SlashBooking** → **Google** → **Configuration OAuth** (bouton  Copier l'URI de redirection). Elle dépend de la valeur de `home_url()` du site.

5. **Create** → Google affiche le **Client ID** et le **Client Secret**. Les conserver précieusement (ils ne seront plus affichés en clair après).

Étape 4 — Connexion dans WordPress

1. WP Admin → SlashBooking → Google → Configuration OAuth.
2. Coller le **Client ID** et le **Client Secret** dans le formulaire → **Enregistrer**.
3. Cliquer **Connecter mon Google Calendar**.
4. Autoriser sur l'écran Google (sélectionner le compte ajouté en testeur à l'étape 2).
5. Retour automatique sur la page admin avec `?connected=1` → un badge  confirme la connexion.
6. **Choisir le calendrier cible** dans la liste déroulante affichée juste sous le badge connexion. Le plugin liste tous les calendriers accessibles au compte ; le calendrier principal est marqué . Cliquer **Enregistrer le calendrier**.

⚠ **Si tu changes de calendrier après coup** : le watch channel et le sync token associés à l'ancien calendrier sont automatiquement réinitialisés. Tu dois cliquer **Démarrer le watch** à nouveau dans la section *Synchronisation entrante* pour activer le push notifications sur le nouveau calendrier. Le `refresh_token` est stocké chiffré (sodium `crypto_secretbox`) dans la table `wp_sb_google_accounts`. La clé de chiffrement provient de la constante `SLASHBOOKING_ENC_KEY` (cf. étape suivante).

Étape 5 — Clé de chiffrement (recommandé)

Ajouter dans `wp-config.php`, **au-dessus** de la ligne `/* That's all, stop editing! */` :

```
define( 'SLASHBOOKING_ENC_KEY', '<64-char hex string>' );
```

Générer une clé :

```
php -r 'echo bin2hex(random_bytes(32));'
```

Sans cette constante, le plugin génère une clé de secours et la stocke dans `wp_options` (`SLASHBOOKING_ENC_KEY_FALLBACK`). Un *admin notice* le rappelle dans le dashboard. **C'est fonctionnel mais moins sûr** : la clé est dans la base, donc accessible à tout backup non chiffré.

⚠ Si tu **changes** la clé après coup, le refresh token déjà stocké devient illisible → reconnexion obligatoire (étape 4.3).

Étape 6 — Sync entrante (webhook + pull)

Une fois OAuth connecté, activer la sync entrante :

1. WP Admin → SlashBooking → Google → Synchronisation entrante.
2. Cliquer **Démarrer le watch**.

Le plugin :

- crée un *channel push notifications* chez Google (TTL 7 jours),
- enregistre un secret HMAC qui valide les notifications entrantes,
- programme un cron quotidien `sb/watch_renew_check` qui renouvelle le channel < 24h avant expiration,
- programme un cron de fallback `sb/google_pull_all` toutes les 15 min (en cas de webhook bloqué par un firewall).

Comment ça marche

1. Tu crées un événement directement dans Google Calendar.
2. Google POST notre webhook : `POST /wp-json/slashbooking/v1/google/webhook`.
3. Le plugin vérifie le header `X-Goog-Channel-Token` (HMAC) → enfile un job `sb/google_pull` (debounce 5 s).
4. Action Scheduler exécute le job → appelle `events.list?syncToken=...` pour récupérer les diffs incrémentaux.
5. Upsert / delete des `BusyBlock` selon le `status` de l'événement (`confirmed` / `tentative` → upsert ; `cancelled` → delete).
6. **Reflection.** Quand notre push (sortant) crée un événement GCal, Google nous re-notifie. On l'ignore via lookup du `google_event_id` dans `wp_sb_bookings`.

Pré-requis réseau

Contrainte	Exigence Google
Protocole	HTTPS uniquement — <code>http://</code> rejeté
Certificat	Valide (pas auto-signé) — Let's Encrypt OK
IP	Publique — RFC1918 (<code>10.*</code> , <code>192.168.*</code>) rejetée
Port	443 standard

En dev local : utiliser un tunnel **ngrok** (`ngrok http 8080`) ou **Cloudflare Tunnel**, et configurer `WP_HOME` / `WP_SITEURL` sur l'URL publique le temps des tests.

Vérification — `wp slashbooking doctor`

```
wp slashbooking doctor
```

Cette commande WP-CLI vérifie en cascade :

1.  Compte connecté + email du compte
2.  Rafraîchissement du token (appel `oauth2/v3/userinfo`)
3.  Liste des calendriers accessibles
4.  Insert + delete d'un événement de test (round-trip API)

5. ✅ Statut watch channel + `expires_at`
6. ✅ Présence d'un `sync_token` valide
7. ✅ Lance un pull de test et rapporte : `upserted / deleted / reflection-ignored`

À lancer après chaque déploiement ou changement de Client ID/Secret.

Diagnostics & monitoring

Interface admin

SlashBooking → Google → Synchronisation entrante affiche :

- Statut du watch (channel id, `expires_at`)
- Date du dernier full sync
- Présence du `sync_token`
- Boutons **Démarrer / Arrêter watch** et **Forcer un pull maintenant**

SlashBooking → Journal affiche tous les événements de sync :

- Filtre `direction=g_to_wp` → opérations entrantes
- Filtre `direction=wp_to_g` → push sortants
- Filtre `entity=watch` → vie du channel (create, renew, stop, expire)
- Filtre `entity=oauth` → tokens (refresh, failure)

CLI

```
# Cron Action Scheduler – voir les jobs en attente
wp action-scheduler run --group=slashbooking

# Crons WP standards
wp cron event list | grep -E "sb_|sb/"

# Pull manuel forcé
wp slashbooking pull
```

Troubleshooting

`oauth_failed` dans `wp slashbooking doctor`

Le refresh token est invalide. Causes possibles :

- Token révoqué côté Google (utilisateur a cliqué *Remove access* dans son compte Google)
- Constante `SLASHBOOKING_ENC_KEY` modifiée
- Mode **Testing** dépassé 7 jours (basculer en Production)

Fix : reconnecter depuis **SlashBooking** → **Google** → **Configuration OAuth** → **Reconnecter mon Google Calendar**.

Le webhook Google n'arrive pas → **BusyBlock** n'apparaît pas

1. Vérifier que le watch est actif (channel id + `expires_at` futur dans l'admin).
2. Vérifier que `home_url()` est en HTTPS :

```
bash wp option get home
```

3. Tester la réception webhook depuis l'extérieur :

```
bash curl -X POST https://votresite.fr/wp-json/slashbooking/v1/google/webhook \
-H "X-Goog-Resource-State: sync" \
-H "X-Goog-Channel-Id: test" \
-H "X-Goog-Channel-Token: <watch_token_secret>"
```

- `200` → OK
 - `401` → token ne correspond pas (re-crée le watch : *Arrêter le watch* puis *Démarrer le watch*)
 - timeout → firewall / WAF bloque (vérifier les logs reverse proxy)
4. Cron fallback : toutes les 15 min, `sb/google_pull_all` fait un pull manuel. Vérifier :

```
bash wp cron event list | grep sb_google_pull_all
```

`syncToken expired` (**410 Gone**) dans le sync log

Normal après une longue période sans pull (Google invalide les tokens > 7 jours). Le **SyncEngine** reset automatiquement le token et refait un full sync au prochain pull. **Aucune action requise**.

`404` ou `410` sur un delete

Tolérés silencieusement : l'événement est déjà absent côté Google, donc l'objectif (le faire disparaître) est atteint. Pas d'entrée d'erreur dans le journal.

Erreurs `5xx` côté Google

Retentées automatiquement par Action Scheduler (backoff exponentiel). Visibles dans le journal jusqu'au succès ou abandon (5 tentatives max). Les `4xx` (sauf 404/410 sur delete) sont consignés sans retry.

`redirect_uri_mismatch` lors de la connexion

L'URL configurée dans Google Cloud Console (étape 3.4) **ne correspond pas exactement** à celle que WordPress envoie. Causes fréquentes :

- `http://` vs `https://`

- `www.` vs sans `www.`
- Slash final
- Site déplacé (`home_url()`) a changé

Fix : copier-coller l'URL **exacte** affichée dans WP Admin → SlashBooking → Google → Configuration OAuth, et la remplacer dans Google Cloud Console.

Désactivation propre

Avant de désactiver le plugin :

1. WP Admin → SlashBooking → Google → Synchronisation entrante → **Arrêter le watch**.

Sinon le channel expire de lui-même en ≤ 7 jours côté Google (pas critique, juste plus propre).

⚠ La désactivation WP n'appelle PAS `stopChannel()` automatiquement — le bootstrap du plugin n'est pas garanti dans le contexte de désactivation, donc on évite les effets de bord.

Annexe — Scopes & permissions

Le plugin demande **uniquement** :

Scope	Usage
<code>https://www.googleapis.com/auth/calendar.events</code>	Créer / modifier / supprimer les événements de RDV
<code>https://www.googleapis.com/auth/calendar.readonly</code>	Lister les calendriers à la connexion (choix cible)

Le plugin **ne lit pas** les événements existants d'autres applications dans le calendrier — il utilise `events.list` filtré par `updatedMin` (delta sync) pour ne traiter que ce qui a changé depuis le dernier pull.

Le plugin **ne stocke pas** le contenu des événements externes : seuls les triplets `(start, end, status)` sont matérialisés en `BusyBlock` pour bloquer les créneaux. Le titre, les invités, etc. restent dans Google.

Document à jour pour slashbooking v1.0.9 — 2026-05-21